NOUS AI ACADEMY / INTENSIVE FILE

TECHNICAL LEARNING FILE

DEEP-11

Preprocessing and Feature Engineering

Leakage-safe transformations

FORMAT	LENGTH	ACCESS
INTENSIVE FILE	50 PAGES	OFFLINE

Edition 2 - original curriculum - editorial review recommended

FILE / ORIENTATION

Purpose

01 FILE GUIDANCE

This optional advanced guide does not gate the core learning path. Apply preprocessing and feature engineering with explicit assumptions, tests, tradeoffs, and limitations.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Prerequisites

01 FILE GUIDANCE

Complete the related core track or pass its diagnostic.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

FILE / ORIENTATION

Study Method

01 FILE GUIDANCE

For each unit, explain the concept, follow the workflow, reproduce the example, analyze a failure, and complete the applied lab. Record evidence locally.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.

UNIT 01 / CONCEPT

Split Before Transform: Concept

01 OBJECTIVE

Explain and apply split before transform using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Split Before Transform is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Split, Before, Transform are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should split before transform be used, and what observable evidence would make that choice defensible? Build a small local example that applies split before transform, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Split Before Transform in one precise sentence and name one assumption.
2. Create a two-step example of split before transform and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKFLOW

Split Before Transform: Workflow

01 OBJECTIVE

Explain and apply split before transform using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Split Before Transform is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to split before transform.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies split before transform, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Split Before Transform in one precise sentence and name one assumption.
2. Create a two-step example of split before transform and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / WORKED EXAMPLE

Split Before Transform: Worked Example

01 OBJECTIVE

Explain and apply split before transform using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies split before transform, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns split before transform into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Split Before Transform in one precise sentence and name one assumption.
2. Create a two-step example of split before transform and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / FAILURE ANALYSIS

Split Before Transform: Failure Analysis

01 OBJECTIVE

Explain and apply split before transform using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how split before transform can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to split before transform. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Split Before Transform in one precise sentence and name one assumption.
2. Create a two-step example of split before transform and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 01 / APPLIED LAB

Split Before Transform: Applied Lab

01 OBJECTIVE

Explain and apply split before transform using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of split before transform into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies split before transform, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Split Before Transform in one precise sentence and name one assumption.
2. Create a two-step example of split before transform and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / CONCEPT

Missingness: Concept

01 OBJECTIVE

Explain and apply missingness using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Missingness is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Missingness are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should missingness be used, and what observable evidence would make that choice defensible? Build a small local example that applies missingness, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Missingness in one precise sentence and name one assumption.
2. Create a two-step example of missingness and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKFLOW

Missingness: Workflow

01 OBJECTIVE

Explain and apply missingness using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Missingness is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to missingness.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies missingness, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Missingness in one precise sentence and name one assumption.
2. Create a two-step example of missingness and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / WORKED EXAMPLE

Missingness: Worked Example

01 OBJECTIVE

Explain and apply missingness using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies missingness, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns missingness into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Missingness in one precise sentence and name one assumption.
2. Create a two-step example of missingness and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / FAILURE ANALYSIS

Missingness: Failure Analysis

01 OBJECTIVE

Explain and apply missingness using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how missingness can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to missingness. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Missingness in one precise sentence and name one assumption.
2. Create a two-step example of missingness and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 02 / APPLIED LAB

Missingness: Applied Lab

01 OBJECTIVE

Explain and apply missingness using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of missingness into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies missingness, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Missingness in one precise sentence and name one assumption.
2. Create a two-step example of missingness and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / CONCEPT

Scaling: Concept

01 OBJECTIVE

Explain and apply scaling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Scaling is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Scaling are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should scaling be used, and what observable evidence would make that choice defensible? Build a small local example that applies scaling, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Scaling in one precise sentence and name one assumption.
2. Create a two-step example of scaling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKFLOW

Scaling: Workflow

01 OBJECTIVE

Explain and apply scaling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Scaling is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to scaling.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies scaling, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Scaling in one precise sentence and name one assumption.
2. Create a two-step example of scaling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / WORKED EXAMPLE

Scaling: Worked Example

01 OBJECTIVE

Explain and apply scaling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies scaling, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns scaling into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Scaling in one precise sentence and name one assumption.
2. Create a two-step example of scaling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / FAILURE ANALYSIS

Scaling: Failure Analysis

01 OBJECTIVE

Explain and apply scaling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how scaling can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to scaling. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Scaling in one precise sentence and name one assumption.
2. Create a two-step example of scaling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 03 / APPLIED LAB

Scaling: Applied Lab

01 OBJECTIVE

Explain and apply scaling using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of scaling into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies scaling, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Scaling in one precise sentence and name one assumption.
2. Create a two-step example of scaling and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / CONCEPT

Categorical Encoding: Concept

01 OBJECTIVE

Explain and apply categorical encoding using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Categorical Encoding is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Categorical, Encoding are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should categorical encoding be used, and what observable evidence would make that choice defensible? Build a small local example that applies categorical encoding, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Categorical Encoding in one precise sentence and name one assumption.
2. Create a two-step example of categorical encoding and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKFLOW

Categorical Encoding: Workflow

01 OBJECTIVE

Explain and apply categorical encoding using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Categorical Encoding is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to categorical encoding.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies categorical encoding, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Categorical Encoding in one precise sentence and name one assumption.
2. Create a two-step example of categorical encoding and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / WORKED EXAMPLE

Categorical Encoding: Worked Example

01 OBJECTIVE

Explain and apply categorical encoding using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies categorical encoding, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns categorical encoding into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Categorical Encoding in one precise sentence and name one assumption.
2. Create a two-step example of categorical encoding and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / FAILURE ANALYSIS

Categorical Encoding: Failure Analysis

01 OBJECTIVE

Explain and apply categorical encoding using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how categorical encoding can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to categorical encoding. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Categorical Encoding in one precise sentence and name one assumption.
2. Create a two-step example of categorical encoding and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 04 / APPLIED LAB

Categorical Encoding: Applied Lab

01 OBJECTIVE

Explain and apply categorical encoding using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of categorical encoding into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies categorical encoding, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Categorical Encoding in one precise sentence and name one assumption.
2. Create a two-step example of categorical encoding and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / CONCEPT

Outliers: Concept

01 OBJECTIVE

Explain and apply outliers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Outliers is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Outliers are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should outliers be used, and what observable evidence would make that choice defensible? Build a small local example that applies outliers, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Outliers in one precise sentence and name one assumption.
2. Create a two-step example of outliers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKFLOW

Outliers: Workflow

01 OBJECTIVE

Explain and apply outliers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Outliers is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to outliers.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies outliers, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Outliers in one precise sentence and name one assumption.
2. Create a two-step example of outliers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / WORKED EXAMPLE

Outliers: Worked Example

01 OBJECTIVE

Explain and apply outliers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies outliers, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns outliers into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Outliers in one precise sentence and name one assumption.
2. Create a two-step example of outliers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / FAILURE ANALYSIS

Outliers: Failure Analysis

01 OBJECTIVE

Explain and apply outliers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how outliers can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to outliers. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Outliers in one precise sentence and name one assumption.
2. Create a two-step example of outliers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 05 / APPLIED LAB

Outliers: Applied Lab

01 OBJECTIVE

Explain and apply outliers using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of outliers into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies outliers, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Outliers in one precise sentence and name one assumption.
2. Create a two-step example of outliers and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / CONCEPT

Text Features: Concept

01 OBJECTIVE

Explain and apply text features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Text Features is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Text, Features are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should text features be used, and what observable evidence would make that choice defensible? Build a small local example that applies text features, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Text Features in one precise sentence and name one assumption.
2. Create a two-step example of text features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKFLOW

Text Features: Workflow

01 OBJECTIVE

Explain and apply text features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Text Features is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to text features.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies text features, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Text Features in one precise sentence and name one assumption.
2. Create a two-step example of text features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / WORKED EXAMPLE

Text Features: Worked Example

01 OBJECTIVE

Explain and apply text features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies text features, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns text features into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Text Features in one precise sentence and name one assumption.
2. Create a two-step example of text features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / FAILURE ANALYSIS

Text Features: Failure Analysis

01 OBJECTIVE

Explain and apply text features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how text features can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to text features. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Text Features in one precise sentence and name one assumption.
2. Create a two-step example of text features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 06 / APPLIED LAB

Text Features: Applied Lab

01 OBJECTIVE

Explain and apply text features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of text features into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies text features, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Text Features in one precise sentence and name one assumption.
2. Create a two-step example of text features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / CONCEPT

Temporal Features: Concept

01 OBJECTIVE

Explain and apply temporal features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Temporal Features is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Temporal, Features are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should temporal features be used, and what observable evidence would make that choice defensible? Build a small local example that applies temporal features, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Temporal Features in one precise sentence and name one assumption.
2. Create a two-step example of temporal features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKFLOW

Temporal Features: Workflow

01 OBJECTIVE

Explain and apply temporal features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Temporal Features is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to temporal features.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies temporal features, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Temporal Features in one precise sentence and name one assumption.
2. Create a two-step example of temporal features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / WORKED EXAMPLE

Temporal Features: Worked Example

01 OBJECTIVE

Explain and apply temporal features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies temporal features, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns temporal features into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Temporal Features in one precise sentence and name one assumption.
2. Create a two-step example of temporal features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / FAILURE ANALYSIS

Temporal Features: Failure Analysis

01 OBJECTIVE

Explain and apply temporal features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how temporal features can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to temporal features. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Temporal Features in one precise sentence and name one assumption.
2. Create a two-step example of temporal features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 07 / APPLIED LAB

Temporal Features: Applied Lab

01 OBJECTIVE

Explain and apply temporal features using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of temporal features into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies temporal features, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Temporal Features in one precise sentence and name one assumption.
2. Create a two-step example of temporal features and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / CONCEPT

Feature Selection: Concept

01 OBJECTIVE

Explain and apply feature selection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Feature Selection is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Feature, Selection are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should feature selection be used, and what observable evidence would make that choice defensible? Build a small local example that applies feature selection, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Feature Selection in one precise sentence and name one assumption.
2. Create a two-step example of feature selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKFLOW

Feature Selection: Workflow

01 OBJECTIVE

Explain and apply feature selection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Feature Selection is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to feature selection.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies feature selection, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Feature Selection in one precise sentence and name one assumption.
2. Create a two-step example of feature selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / WORKED EXAMPLE

Feature Selection: Worked Example

01 OBJECTIVE

Explain and apply feature selection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies feature selection, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns feature selection into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Feature Selection in one precise sentence and name one assumption.
2. Create a two-step example of feature selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / FAILURE ANALYSIS

Feature Selection: Failure Analysis

01 OBJECTIVE

Explain and apply feature selection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how feature selection can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to feature selection. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Feature Selection in one precise sentence and name one assumption.
2. Create a two-step example of feature selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 08 / APPLIED LAB

Feature Selection: Applied Lab

01 OBJECTIVE

Explain and apply feature selection using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of feature selection into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies feature selection, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Feature Selection in one precise sentence and name one assumption.
2. Create a two-step example of feature selection and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / CONCEPT

Pipeline Testing: Concept

01 OBJECTIVE

Explain and apply pipeline testing using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Pipeline Testing is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. Begin by asking what problem the idea solves, what information enters the process, what result should leave it, and what evidence would show that the result is useful. In Preprocessing and Feature Engineering, the terms Pipeline, Testing are not vocabulary to memorize in isolation; they describe a system of relationships. A strong learner can translate the idea between plain language, a diagram, a small numerical or code example, and a statement of limitations. This page establishes that translation before adding detail.

03 REASONING

Central question: When should pipeline testing be used, and what observable evidence would make that choice defensible? Build a small local example that applies pipeline testing, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Pipeline Testing in one precise sentence and name one assumption.
2. Create a two-step example of pipeline testing and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKFLOW

Pipeline Testing: Workflow

01 OBJECTIVE

Explain and apply pipeline testing using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Pipeline Testing is a central part of preprocessing and feature engineering because it changes how inputs, assumptions, implementation choices, and evidence connect. A first-principles view separates the mechanism into state, operation, constraint, and test. State describes what is currently known. The operation transforms that state. A constraint rules out invalid moves, and a test compares the output with an expectation. This structure prevents an implementation from becoming a collection of unexplained steps. It also makes debugging local: identify which state, operation, constraint, or test failed. Apply this model directly to pipeline testing.

03 REASONING

Workflow: define the smallest valid input; predict the next state; perform one operation; test one invariant; then expand. Example: Build a small local example that applies pipeline testing, records the result, and tests one failure condition.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Pipeline Testing in one precise sentence and name one assumption.
2. Create a two-step example of pipeline testing and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / WORKED EXAMPLE

Pipeline Testing: Worked Example

01 OBJECTIVE

Explain and apply pipeline testing using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Consider this task: Build a small local example that applies pipeline testing, records the result, and tests one failure condition. First state the expected output before calculating or coding. Next choose a minimal representation and execute one step at a time. After every step, compare the intermediate state with the rule that should still hold. Finally, test a boundary case that differs from the example in exactly one important way. This worked-example pattern turns pipeline testing into a repeatable method rather than a memorized answer.

03 REASONING

Worked reasoning: 1) restate the task; 2) identify knowns and unknowns; 3) choose the smallest method; 4) calculate or trace; 5) verify independently; 6) explain what would make the result fail.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Pipeline Testing in one precise sentence and name one assumption.
2. Create a two-step example of pipeline testing and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / FAILURE ANALYSIS

Pipeline Testing: Failure Analysis

01 OBJECTIVE

Explain and apply pipeline testing using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Failure analysis asks how pipeline testing can produce a plausible but wrong result. Common causes include an incorrect assumption, a representation that loses information, an edge case omitted from tests, an invalid comparison, or evidence taken from the wrong stage of the process. Change one factor at a time, preserve the failing example, and write a regression test before repairing the implementation. A clear failure record is part of the learning artifact.

03 REASONING

Failure probe: construct the smallest input that breaks the naive approach to pipeline testing. State the expected behavior and why the naive result is misleading.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Pipeline Testing in one precise sentence and name one assumption.
2. Create a two-step example of pipeline testing and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

UNIT 09 / APPLIED LAB

Pipeline Testing: Applied Lab

01 OBJECTIVE

Explain and apply pipeline testing using explicit inputs, assumptions, steps, evidence, and failure checks.

02 CORE EXPLANATION

Implementation converts the idea of pipeline testing into inspectable behavior. Start with a deterministic example small enough to check by hand. Keep data preparation, core logic, and reporting separate. Add assertions for shape, type, range, or state as appropriate. Record the configuration and avoid relying on hidden global state. The goal is not merely to obtain output; it is to create an output whose origin can be explained and reproduced.

03 REASONING

Lab brief: Build a small local example that applies pipeline testing, records the result, and tests one failure condition. Save the input, expected output, observed output, and a short explanation of any difference.

04 IMPLEMENTATION

```
split data before fitting transforms
fit preprocessing on training data only
fit a simple baseline
evaluate with the chosen metric
inspect errors by meaningful group
record configuration and random seed
```

05 PRACTICE

1. Define Pipeline Testing in one precise sentence and name one assumption.
2. Create a two-step example of pipeline testing and verify the intermediate state.
3. Name one boundary case, one likely misconception, and one test that would expose it.
4. Connect this page to a prior concept and explain why the connection matters.

FILE / ORIENTATION

Deep-Dive Capstone

01 FILE GUIDANCE

Build a compact, reproducible artifact demonstrating preprocessing and feature engineering. Include assumptions, a baseline, tests, failure analysis, results, limitations, and instructions for repeating the work offline.

02 LOCAL USE

Index this page in the offline database and preserve its resource and page identifiers for bookmarks, notes, highlights, and progress.